August 24, 2026 at 11:59

1. Introduction. This program counts **open** knight's tours (open Hamiltonian paths) of the $m \times n$ knight graph by a **dual-frontier** transfer that is aggregated with **bucket sorting** rather than a trie, so it parallelizes; and it extracts the **periodic** transfer once the frontier stabilizes, so that far columns are reached by a cheap sparse matrix-vector product (SpMV) instead of re-running the generator.

The method is Knuth's (as in `DYNAHAM/dynahamp`), reimplemented from scratch. Work in the augmented graph $G^+ = G + K_1$: a new **apex** vertex joins every cell, and an open tour of G is a Hamiltonian cycle of G^+ through the apex. Place the cells column by column. After s cells are placed, the **frontier** is the set of not-yet-placed vertices adjacent to a placed one, plus the apex. Each frontier vertex is **bare** (degree 0), **outer** (degree 1, a subpath endpoint), or **inner** (degree 2). The state is that pattern together with the pairing of the outer endpoints. Because it is coded by **relative** positions, not absolute vertex numbers, the code is translation-invariant: the sorted state set repeats with some small period once we are in the bulk, which is exactly what makes the transfer periodic and the SpMV possible.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

⟨ Types 2 ⟩
⟨ Globals 3 ⟩
⟨ Subroutines 4 ⟩
⟨ Main 19 ⟩

2. Board and frontier. Cell (r, c) is vertex $v = cm + r$; there are $V = mn$ cells and the apex is V. *frontier_before*(s) lists (sorted) the frontier just before cell s: the apex, s itself (an "extended" frontier, always present), and each future cell $> s$ with an already-placed neighbour.

⟨ Types 2 ⟩ ≡
```
typedef unsigned long long u64;
```
See also section 7.

This code is used in section 1.

3.
```
#define MAXF 512
#define MAXLEV 40
```
⟨ Globals 3 ⟩ ≡
```
int m, n, V;
int NB[64 * 64][8], ND[64 * 64];
static const int KR[8] = {-2, -2, -1, -1, 1, 1, 2, 2};
static const int KC[8] = {-1, 1, -2, 2, -2, 2, -1, 1};
int fr[MAXF], ifrb[64 * 64 + 2];
```
See also sections 5, 8, and 10.

This code is used in section 1.

4. ⟨ Subroutines 4 ⟩ ≡

```
void build_board(void)
{
  int r, c, k;
  V = m * n;
  for (c = 0; c < n; c++)
    for (r = 0; r < m; r++) {
      int v = c * m + r;
      ND[v] = 0;
      for (k = 0; k < 8; k++) {
        int rr = r + KR[k], cc = c + KC[k];
        if (rr ≥ 0 ∧ rr < m ∧ cc ≥ 0 ∧ cc < n) NB[v][ND[v]++] = cc * m + rr;
      }
    }
}
int frontier_before(int s)
{
  int q = 0, v, u, k;
  static char inF[64 * 64 + 2];
  for (v = 0; v ≤ V; v++) inF[v] = 0;
  inF[V] = 1;
  if (s < V) inF[s] = 1;
  for (v = s + 1; v < V; v++)
    for (k = 0; k < ND[v]; k++) {
      u = NB[v][k];
      if (u < s) {
        inF[v] = 1;
        break;
      }
    }
  for (v = 0; v ≤ V; v++)
    if (inF[v]) {
      fr[q] = v;
      ifrb[v] = q;
      q++;
    }
  return q;
}
```

See also sections 6, 9, 11, 12, and 16.

This code is used in section 1.

5. Mate table, derived edges, key, completion. The state is *mate*[ ] over frontier positions: -2 bare, -1 inner, else the partner position (outer). *add_derived*(i, j) joins the subpaths ending at i, j; if they are the two ends of one subpath a cycle forms (flagged in *cycle*, both ends set inner). The key is one byte per position; equal (also column-shifted) patterns give equal keys. A cycle through the apex is a valid open m'-tour iff the covered (inner) cells are the contiguous prefix $\{0..m'-1\}$ and the rest of the board frontier is bare; *completion_mp* returns that m'.

```
⟨ Globals 3 ⟩ +≡
  int mate[MAXF];
  int cycle;
```

6. ⟨ Subroutines 4 ⟩ +≡

```
int add_derived(int i, int j)
{
  cycle = 0;
  if (mate[i] ≡ −1 ∨ mate[j] ≡ −1) return 0;
  if (mate[i] ≡ −2) {
    if (mate[j] ≡ −2) {
      if (i ≡ j) {
        cycle = 1;
        return 0;
      }
      mate[i] = j;
      mate[j] = i;
      return 1;
    }
    mate[i] = mate[j];
    mate[mate[j]] = i;
    mate[j] = −1;
    return 1;
  }
  else if (mate[j] ≡ −2) {
    mate[j] = mate[i];
    mate[mate[i]] = j;
    mate[i] = −1;
    return 1;
  }
  else if (mate[i] ≠ j) {
    mate[mate[i]] = mate[j];
    mate[mate[j]] = mate[i];
    mate[i] = mate[j] = −1;
    return 1;
  }
  mate[i] = mate[j] = −1;
  cycle = 1;
  return 0;
}
int keyof(int q, unsigned char *key)
{
  int i;
  for (i = 0; i < q; i++) key[i] = mate[i] ≡ −2 ? 0 : mate[i] ≡ −1 ? 255 : (unsigned char)(1 + mate[i]);
  return q;
}
int completion_mp(int q, int apexpos, int *frn, int s)
{
  int k, mp = s + 1;
  if (mate[apexpos] ≠ −1) return 0;
  k = 0;
  while (k < q ∧ frn[k] ≠ V ∧ mate[k] ≡ −1 ∧ frn[k] ≡ mp) {
    mp++;
    k++;
  }
```

```
  for ( ; k < q; k++) {
    if (frn[k] ≡ V) continue;
    if (mate[k] ≠ −2) return 0;
  }
  return mp;
}
```

7. Bucket aggregation. Each step emits successor records; we sort by key and merge equal keys. *src* carries the emitting state's index, used only while recording the edge tables.

```
⟨ Types 2 ⟩ +≡
  typedef struct {
    long off;
    int len;
    u64 w;
    long src;
  } Rec;
```

8.
```
⟨ Globals 3 ⟩ +≡
  unsigned char *kp;
  long kcap, kuse;
  Rec *rc;
  long nr, rcap;
  unsigned char *cb;
  int rcmp(const void *A, const void *B)
  {
    const Rec *a = A, *b = B;
    int l = a->len < b->len ? a->len : b->len;
    int d = memcmp(cb + a->off, cb + b->off, l);
    return d ? d : a->len - b->len;
  }
```

9.
```
⟨ Subroutines 4 ⟩ +≡
  void emit(unsigned char *key, int len, u64 w, long src)
  {
    if (kuse + len > kcap) {
      kcap = kcap * 2 + len + 65536;
      kp = realloc(kp, kcap);
    }
    if (nr ≥ rcap) {
      rcap = rcap * 2 + 65536;
      rc = realloc(rc, rcap * sizeof(Rec));
    }
    memcpy(kp + kuse, key, len);
    rc[nr].off = kuse;
    rc[nr].len = len;
    rc[nr].w = w;
    rc[nr].src = src;
    kuse += len;
    nr++;
  }
```

10. The transfer step. The bucket holds states as (key,weight) over the previous frontier. To place cell s: build the base mate table *bmate* over the new frontier (survivors carried, newcomers bare) with s in a temporary slot, then bring s to degree 2 by adding its $2 - \deg(s)$ edges to future neighbours or the apex. When *rec* is set we also record the integer edge table (src index → dst index) and the completion contributions for this level.

```
⟨ Globals 3 ⟩ +≡
  unsigned char *curkp;
  long *curoff;
  int *curkl;
  u64 *curw;
  long ncur;
  u64 cnt[1 ≪ 16];
  int qnew, posS, apexnew, STEMP;
  int bmate[MAXF], o2n[MAXF];
  int recording, reclev;
  typedef struct {
    int src, dst;
    u64 c;
  } Edge;
  typedef struct {
    int src, delta;
    u64 mult;
  } Comp;
  Edge *edges[MAXLEV];
  long nedge[MAXLEV];
  Comp *comps[MAXLEV];
  long ncomp[MAXLEV];
  long nstate[MAXLEV + 1];
  int Plevs;
  u64 *seedv;
  long seedn;
  Comp reccomp_buf[1 ≪ 20];
  long reccomp_n;
  int ecmp(const void *A, const void *B)
  {
    const Edge *a = A, *b = B;
    if (a→src ≠ b→src) return a→src − b→src;
    return a→dst − b→dst;
  }
```

11. ⟨ Subroutines 4 ⟩ +≡

```
void build_bmate(int *omate, int qold)
{
  int i;
  for (i = 0; i ≤ qnew; i++) bmate[i] = −2;
  STEMP = qnew;
  for (i = 0; i < qold; i++) {
    int dst = (i ≡ posS) ? STEMP : o2n[i];
    if (dst < 0) continue;
    if (omate[i] ≡ −1) bmate[dst] = −1;
    else if (omate[i] ≥ 0) {
      int op = omate[i];
      int pdst = (op ≡ posS) ? STEMP : o2n[op];
      bmate[dst] = pdst ≥ 0 ? pdst : −2;
    }
  }
}
```

12. ⟨ Subroutines 4 ⟩ +≡

```
void run_step(int s, int rec)
{
  int i;
  long si;
  long in_ncur = ncur;
  int qold = frontier_before(s);
  posS = ifrb[s];
  static int frold[MAXF];
  for (i = 0; i < qold; i++) frold[i] = fr[i];
  qnew = frontier_before(s + 1);
  static int ifrnew[64 * 64 + 2], frnew[MAXF];
  for (i = 0; i < qnew; i++) frnew[i] = fr[i];
  for (i = 0; i ≤ V; i++) ifrnew[i] = −1;
  for (i = 0; i < qnew; i++) ifrnew[frnew[i]] = i;
  apexnew = ifrnew[V];
  int nbr[16], rr = 0;
  nbr[rr++] = apexnew;
  for (i = 0; i < ND[s]; i++) {
    int w = NB[s][i];
    if (w > s ∧ ifrnew[w] ≥ 0) nbr[rr++] = ifrnew[w];
  }
  for (i = 0; i < qold; i++) o2n[i] = (frold[i] ≡ s) ? −1 : ifrnew[frold[i]];
  nr = 0;
  kuse = 0;
  reccomp_n = 0;
  ⟨ Expand every state at cell s 13 ⟩;
  ⟨ Sort, reduce, and (if recording) capture the edge table 15 ⟩;
}
```

13. ⟨ Expand every state at cell s 13 ⟩ ≡

```
for (si = 0; si < ncur; si++) {
  unsigned char *ok = curkp + curoff[si];
  int okl = curkl[si];
  u64 w = curw[si];
  static int omate[MAXF];
  int a, b;
  for (i = 0; i < okl; i++) {
    int c = ok[i];
    omate[i] = c ≡ 0 ? -2 : c ≡ 255 ? -1 : c - 1;
  }
  int deg = omate[posS] ≡ -2 ? 0 : omate[posS] ≡ -1 ? 2 : 1, need = 2 - deg;
  unsigned char nk[MAXF];
  int nl;
  if (need ≡ 0) {
    build_bmate(omate, qold);
    for (i = 0; i < qnew; i++) mate[i] = bmate[i];
    nl = keyof(qnew, nk);
    emit(nk, nl, w, si);
  }
  else if (need ≡ 1) {
    for (a = 0; a < rr; a++) {
      build_bmate(omate, qold);
      for (i = 0; i ≤ qnew; i++) mate[i] = bmate[i];
      if (add_derived(STEMP, nbr[a])) {
        nl = keyof(qnew, nk);
        emit(nk, nl, w, si);
      }
      else if (cycle) {
        int mp = completion_mp(qnew, apexnew, frnew, s);
        if (mp) {
          cnt[mp] += w;
          ⟨ Record a completion 14 ⟩;
        }
      }
    }
  }
  else {
    for (a = 0; a < rr; a++)
      for (b = a + 1; b < rr; b++) {
        build_bmate(omate, qold);
        for (i = 0; i ≤ qnew; i++) mate[i] = bmate[i];
        if (¬add_derived(STEMP, nbr[a])) {
          if (cycle) {
            int mp = completion_mp(qnew, apexnew, frnew, s);
            if (mp) {
              cnt[mp] += w;
              ⟨ Record a completion 14 ⟩;
            }
          }
          continue;
```

```
        }
        if (add_derived(STEMP, nbr[b])) {
          nl = keyof(qnew, nk);
          emit(nk, nl, w, si);
        }
        else if (cycle) {
          int mp = completion_mp(qnew, apexnew, frnew, s);
          if (mp) {
            cnt[mp] += w;
            ⟨ Record a completion 14 ⟩;
          }
        }
      }
    }
  }
```

This code is used in section 12.

14. ⟨ Record a completion 14 ⟩ ≡

```
if (rec) {
  reccomp_buf[reccomp_n].src = si;
  reccomp_buf[reccomp_n].delta = mp − (s + 1);
  reccomp_buf[reccomp_n].mult = 1;
  reccomp_n++;
}
```

This code is used in section 13.

15. The reduce also, when recording, resolves each emitted record's destination to its bucket index, giving integer edges (src, dst, coeff).

⟨ Sort, reduce, and (if recording) capture the edge table 15 ⟩ ≡

```
{
  cb = kp;
  qsort(rc, nr, sizeof(Rec), rcmp);
  curoff = realloc(curoff, (nr + 1) * sizeof(long));
  curkl = realloc(curkl, (nr + 1) * sizeof(int));
  curw = realloc(curw, (nr + 1) * sizeof(u64));
  curkp = realloc(curkp, kuse + 1);
  Edge *eb = 0;
  long enb = 0;
  if (rec) eb = malloc((nr + 1) * sizeof(Edge));
  long k = 0, use = 0, ri;
  for (ri = 0; ri < nr; ) {
    long j = ri + 1;
    u64 sw = rc[ri].w;
    if (rec) {
      long t;
      for (t = ri; t < nr; t++) {
        if (t > ri ∧ ¬(rc[t].len ≡ rc[ri].len ∧ memcmp(kp + rc[t].off, kp + rc[ri].off, rc[ri].len) ≡ 0))
          break;
        eb[enb].src = (int) rc[t].src;
        eb[enb].dst = (int) k;
        eb[enb].c = 1;
        enb++;
      }
    }
    while (j < nr ∧ rc[j].len ≡ rc[ri].len ∧ memcmp(kp + rc[j].off, kp + rc[ri].off, rc[ri].len) ≡ 0) {
      sw += rc[j].w;
      j++;
    }
    memcpy(curkp + use, kp + rc[ri].off, rc[ri].len);
    curoff[k] = use;
    curkl[k] = rc[ri].len;
    curw[k] = sw;
    use += rc[ri].len;
    k++;
    ri = j;
  }
  ncur = k;
  if (rec) {
    long o = 0, p;
    qsort(eb, enb, sizeof(Edge), ecmp);
    for (p = 0; p < enb; ) {
      long q2 = p + 1;
      u64 cc = 1;
      while (q2 < enb ∧ eb[q2].src ≡ eb[p].src ∧ eb[q2].dst ≡ eb[p].dst) {
        cc++;
        q2++;
```

```
        }
        eb[o] = eb[p];
        eb[o].c = cc;
        o++;
        p = q2;
      }
      edges[reclev] = realloc(eb, (o + 1) * sizeof(Edge));
      nedge[reclev] = o;
      nstate[reclev] = in_ncur;
      nstate[reclev + 1] = ncur;
      comps[reclev] = malloc((reccomp_n + 1) * sizeof(Comp));
      memcpy(comps[reclev], reccomp_buf, reccomp_n * sizeof(Comp));
      ncomp[reclev] = reccomp_n;
      reclev++;
    }
  }
```

This code is used in section 12.

16. Driver: build, extract the period, then SpMV. Sweep an $m \times W_b$ strip; when the boundary key-set (fingerprinted) repeats with period $p \in \{1, 2\}$ at column c_0, record the next p columns' edge tables and capture the seed vector. Then iterate the SpMV to reach column N: at each recorded level apply the edge table to the state vector and add the level's completions to *cnt2*. The two agree on every column the SpMV fully covers ($c \geq c_0 + 3$), and the SpMV alone yields the columns past W_b.

```
⟨ Subroutines 4 ⟩ +≡
  void seed_bucket(void)
  {
    int i;
    int q = frontier_before(0);
    for (i = 0; i < q; i++) mate[i] = -2;
    unsigned char key[MAXF];
    int kl = keyof(q, key);
    curkp = malloc(1 ≪ 20);
    curoff = malloc(sizeof(long));
    curkl = malloc(sizeof(int));
    curw = malloc(sizeof(u64));
    memcpy(curkp, key, kl);
    curoff[0] = 0;
    curkl[0] = kl;
    curw[0] = 1;
    ncur = 1;
  }
  u64 cnt2[1 ≪ 16];
  int run_periodic(int mm, int Wb, int Next)
  {
    int i, s, c;
    m = mm;
    n = Wb;
    build_board( );
    memset(cnt, 0, sizeof (cnt));
    memset(cnt2, 0, sizeof (cnt2));
    recording = 0;
    reclev = 0;
    seed_bucket( );
    static u64 colfp[4096];
    int c0 = -1, period = 0, recstart = -1, recend = -1;
    for (s = 0; s < V; s++) {
      if (recording ∧ s ≡ recstart) {
        seedn = ncur;
        seedv = malloc(ncur * sizeof(u64));
        for (i = 0; i < ncur; i++) seedv[i] = curw[i];
        nstate[0] = ncur;
        reclev = 0;
      }
      run_step(s, recording ∧ s ≥ recstart ∧ s ≤ recend);
      if (s % m ≡ m − 1) {
        c = s/m;
        u64 h = 14695981039346656037ULL;
        long t;
```

```
      for (t = 0; t < ncur; t++) {
        unsigned char *kk = curkp + curoff[t];
        int L = curkl[t], z;
        for (z = 0; z < L; z++) {
          h ⊕= kk[z];
          h *= 1099511628211ULL;
        }
        h ⊕= #9e;
        h *= 1099511628211ULL;
      }
      colfp[c] = h;
      if (c0 < 0) {
        if (c ≥ 1 ∧ colfp[c] ≡ colfp[c − 1]) {
          period = 1;
          c0 = c;
        }
        else if (c ≥ 2 ∧ colfp[c] ≡ colfp[c − 2]) {
          period = 2;
          c0 = c;
        }
        if (c0 ≥ 0) {
          recstart = (c0 + 1) * m;
          recend = (c0 + 1 + period) * m − 1;
          Plevs = period * m;
          recording = 1;
          fprintf(stderr, "stable␣col␣%d,␣period␣%d\n", c0, period);
        }
      }
    }
  }
  ⟨ Iterate the SpMV out to column N 17 ⟩;
  ⟨ Report and cross-check 18 ⟩;
  return c0;
}
```

17. ⟨ Iterate the SpMV out to column N 17 ⟩ ≡

```
{
  u64 *v = malloc(nstate[0] * sizeof(u64));
  memcpy(v, seedv, nstate[0] * sizeof(u64));
  int basecol = c0 + 1;
  while (basecol ≤ Next) {
    int L;
    for (L = 0; L < Plevs; L++) {
      int abscol = basecol + L/m, substep = L % m;
      long e;
      for (e = 0; e < ncomp[L]; e++) {
        int idx = abscol * m + substep + 1 + comps[L][e].delta;
        cnt2[idx] += v[comps[L][e].src] * comps[L][e].mult;
      }
      u64 *vn = calloc(nstate[L + 1], sizeof(u64));
      for (e = 0; e < nedge[L]; e++) vn[edges[L][e].dst] += v[edges[L][e].src] * edges[L][e].c;
      free(v);
      v = vn;
    }
    basecol += period;
  }
  free(v);
}
```

This code is used in section 16.

18. ⟨ Report and cross-check 18 ⟩ ≡

```
{
  int good = 1, cc;
  for (cc = c0 + 3; cc ≤ Next ∧ cc ≤ Wb; cc++)
    if (cnt[cc * m] ≠ cnt2[cc * m]) {
      good = 0;
      fprintf(stderr, "MISMATCH␣c=%d␣direct=%llu␣spmv=%llu\n", cc, cnt[cc * m], cnt2[cc * m]);
    }
  for (cc = 1; cc ≤ Next; cc++) {
    u64 val = cc ≤ Wb ? cnt[cc * m] : cnt2[cc * m];
    if (val) printf("open␣%dx%d␣=␣%llu%s\n", m, cc, val, cc > Wb ? "␣␣(SpMV)" : "");
  }
  fprintf(stderr, "%s\n", good ? "SpMV␣matches␣direct" : "SpMV␣MISMATCH");
}
```

This code is used in section 16.

19. Main. No arguments: self-checks (direct sweep vs known values, and SpMV vs direct). *dualhamm WbN*: build to column W_b, then extend to column N by SpMV.

```
⟨ Main 19 ⟩ ≡
  int main(int argc, char *argv[])
  {
    if (argc ≡ 4) {
      run_periodic(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]));
      return 0;
    }
    struct {
      int m, Wb, c;
      u64 e;
    } chk[] = {{5, 12, 4, 82}, {5, 12, 6, 18784}, {5, 12, 8, 18061054ULL}, {5, 12, 9, 264895640ULL}, {5, 12, 10,
        7886117822ULL}, {7, 6, 4, 6378}, {6, 7, 6, 3318960}, {0, 0, 0, 0}};
    int i, bad = 0, lm = 0, lw = 0;
    for (i = 0; chk[i].m; i++) {
      if (chk[i].m ≠ lm ∨ chk[i].Wb ≠ lw) {
        run_periodic(chk[i].m, chk[i].Wb, chk[i].Wb);
        lm = chk[i].m;
        lw = chk[i].Wb;
      }
      u64 g = cnt[chk[i].c * chk[i].m];
      printf("open %dx%d = %llu  exp %llu  %s\n", chk[i].m, chk[i].c, g, chk[i].e,
          g ≡ chk[i].e ? "OK" : "FAIL");
      if (g ≠ chk[i].e) bad++;
    }
    printf("%s\n", bad ? "SOME FAILED" : "ALL OK");
    return bad ? 1 : 0;
  }
```

This code is used in section 1.

A: 8, 10.
a: 8, 10, 13.
abscol: 17.
add_derived: 5, 6, 13.
apexnew: 10, 12, 13.
apexpos: 6.
argc: 19.
argv: 19.
atoi: 19.
B: 8, 10.
b: 8, 10, 13.
bad: 19.
basecol: 17.
bmate: 10, 11, 13.
build_bmate: 11, 13.
build_board: 4, 16.
c: 4, 10, 13, 16, 19.
calloc: 17.
cb: 8, 15.
cc: 4, 15, 18.
chk: 19.
cnt: 10, 13, 16, 18, 19.
cnt2: 16, 17, 18.
colfp: 16.
Comp: 10, 15.
completion_mp: 5, 6, 13.
comps: 10, 15, 17.
curkl: 10, 13, 15, 16.
curkp: 10, 13, 15, 16.
curoff: 10, 13, 15, 16.
curw: 10, 13, 15, 16.
cycle: 5, 6, 13.
c0: 16, 17, 18.
d: 8.
deg: 13.
delta: 10, 14, 17.
dst: 10, 11, 15, 17.
dualham: 19.
e: 17, 19.
eb: 15.

⟨ Expand every state at cell s 13 ⟩ Used in section 12.
⟨ Globals 3, 5, 8, 10 ⟩ Used in section 1.
⟨ Iterate the SpMV out to column N 17 ⟩ Used in section 16.
⟨ Main 19 ⟩ Used in section 1.
⟨ Record a completion 14 ⟩ Used in section 13.
⟨ Report and cross-check 18 ⟩ Used in section 16.
⟨ Sort, reduce, and (if recording) capture the edge table 15 ⟩ Used in section 12.
⟨ Subroutines 4, 6, 9, 11, 12, 16 ⟩ Used in section 1.
⟨ Types 2, 7 ⟩ Used in section 1.

DUALHAM